Implementação de Biblioteca de Pré-processamento Nativa em Python: Uma Aplicação em Dados do Spotify

Chaira S. Sacramento[1], Darlan Deivid S. Silva[2], Dylan C. Soares[1], Leonardo F. C. Nery[3]

[1]Sistemas de informação – Centro Universitário de Excelência de Feira de Santana (UNEX)
Av. Artêmia Pires Freitas, s/n - Sim, 44085-370 – Feira de Santana – BA – Brazil

chairassacramento@gmail.com,
darlandeivid64@gmail.com,dylancoelhosoares@gmail.com,
leoneryfreitas@gmail.com

Abstract. *This paper documents the second stage of the Dendezeiro trainee program, focused on the development of a proprietary data preprocessing library in Python. Without the aid of external packages, modules were implemented for handling missing values, numerical scaling, and encoding of categorical variables. Validation occurred through automated unit tests and the application of the library to a real Spotify dataset, evidencing the architecture's robustness in data sanitization and preparation for future recommendation algorithms.*

Resumo. *Este artigo documenta a segunda etapa do programa de trainee Dendezeiro, focada no desenvolvimento de uma biblioteca proprietária de pré-processamento de dados em Python. Sem o auxílio de pacotes externos, foram implementados módulos para tratamento de valores ausentes, escalonamento numérico e codificação de variáveis categóricas. A validação ocorreu através de testes unitários automatizados e da aplicação da biblioteca em um dataset real do Spotify, evidenciando a robustez da arquitetura na higienização e adequação dos dados para futuros algoritmos de recomendação.*

1. Introdução

Antes da construção de qualquer modelo de aprendizado de máquina, é imperativo garantir que os dados de alimentação sejam de alta qualidade. O pré-processamento transforma dados brutos e ruidosos em formatos limpos, consistentes e matematicamente compreensíveis pelos algoritmos. Este relatório detalha a implementação nativa das classes Preprocessing, MissingValueProcessor, Scaler e Encoder, validando-as sob o rigor de testes unitários e em um cenário real com mais de 8 mil registros.

Estruturamos o restante deste nosso relatório da seguinte forma: na Seção 2, apresentamos a nossa fundamentação teórica com os conceitos e exemplos das técnicas utilizadas. Na Seção 3, detalhamos a metodologia da nossa implementação. Na Seção 4, discutimos os resultados dos nossos testes unitários e a aplicação no dataset. Por fim, na Seção 5, trazemos as nossas considerações finais, desafios e pontos de melhoria.

2. Fundamentação Teórica

O pré-processamento é amplamente reconhecido na literatura de Ciência de Dados como a etapa mais crítica do fluxo de descoberta de conhecimento **(Han et al., 2011; Faceli et al., 2021)**. Para atender a este escopo, nós fundamentamos a nossa biblioteca em três pilares teóricos:

- **Tratamento de Valores Ausentes (Missing Values):** Bancos de dados reais frequentemente contêm ruídos ou campos não preenchidos. As abordagens teóricas consistem na exclusão de instâncias (*dropna*) ou na imputação de valores (*fillna*). **Como exemplo prático**, se em nosso dataset uma música não possuir o nome do artista (valor None), nós podemos remover essa linha inteira (*dropna*) ou substituir o nulo pelo texto 'Artista Desconhecido' (*fillna*), permitindo que o algoritmo continue rodando.
- **Escalonamento Numérico (Scaling):** Variáveis com grandezas muito discrepantes podem enviesar modelos algorítmicos. O **Min-Max Scaler** normaliza os dados proporcionalmente para um intervalo fixo (usualmente de 0 a 1). Já a padronização via **Z-score** converte os dados de modo que a distribuição tenha média 0 e desvio padrão 1. **Como exemplo prático**, nós aplicamos o Min-Max para transformar a 'Popularidade' de uma faixa (que vai de 0 a 100) em uma escala de 0.0 a 1.0. Isso impede que a popularidade tenha um peso matemático injustamente maior do que a variável 'Duração', apenas por ter valores absolutos maiores.
- **Codificação Categórica (Encoding):** Atributos nominais devem ser convertidos em vetores. O **Label Encoding** estabelece uma relação biunívoca entre uma categoria e um índice inteiro. O **One-Hot Encoding** expande cada categoria em uma variável *dummy* independente. **Como exemplo prático**, no Label Encoding, nós pegamos a categoria 'Tipo de Álbum' e transformamos 'Album' em 0 e 'Single' em 1. Se usássemos o One-Hot, nós criaríamos duas colunas separadas: 'é_album' e 'é_single', preenchidas com 1 (sim) ou 0 (não).

3. Metodologia

A implementação foi realizada em linguagem Python pura **(Python Software Foundation, 2024)**, utilizando o paradigma de Orientação a Objetos. O desenvolvimento absteve-se de frameworks de alto nível, priorizando a manipulação direta de dicionários e listas nativas. O desenvolvimento seguiu a seguinte lógica computacional:

3.1. Validação de Estrutura de Dados Para garantir a integridade do pipeline, a classe

```python
tamanho_base = len(next(iter(self.dataset.values())))

        for col_name, col_data in self.dataset.items():
            if len(col_data) != tamanho_base:
                raise ValueError(f"As colunas têm tamanhos diferentes. Coluna '{col_name}' tem {len(col_data)} itens.")
```

Figura 1. Validação de cardinalidade do dataset.

base Preprocessing implementou uma validação rígida de formato (*shape*). Foi necessário garantir que todas as colunas possuíssem a mesma cardinalidade:

A utilização de um iterador garante complexidade temporal $O(1)$ para capturar o tamanho de referência.

3.2. Processamento de Valores Ausentes (Missing Values) O tratamento de nulos foi isolado na classe MissingValueProcessor. A lógica principal mapeia os índices e reconstrói um novo dicionário de dados limpos, evitando problemas de reindexação dinâmica que ocorreriam ao apagar elementos durante um laço de repetição:

```python
index_nulo = []

        # Percorre o dataset linha por linha
        for i in range(row_count):
            # Para cada linha, verifica as colunas alvo
            for col in target_cols:
                # Se eu encontrar um nulo nesta linha, guarda o índice e pulo para a próxima
                if self.dataset[col][i] is None:
                    index_nulo.append(i)
                    break
```

Figura 2. Mapeamento de linhas nulas na classe MissingValueProcessor.

O método dropna utiliza este mapeamento prévio para reconstruir o dataset ignorando as linhas defeituosas de forma performática.

3.3. Tratamento e Escalonamento Numérico (Módulo Scaler) O Min-Max Scaler redimensiona os valores proporcionalmente com base em seus extremos. O código incorpora tratamento matemático para evitar divisão por zero em cenários de variância nula e ignora dados vazios (None) dinamicamente:

```
valor_min = min(valores_validos)
            valor_max = max(valores_validos)

            denominador = valor_max - valor_min

            nova_lista = [] #Criar uma nova lista

            #Percorre por cada número da lista original
            for valor in dados_coluna:

                if valor is None:
                    nova_lista.append(None) #Deixa o vazio como vazio
                else:
                    if denominador == 0:
                        novo_valor = 0.0
                    else:
                        novo_valor = (valor - valor_min) / denominador
```

Figura 3. Lógica matemática de normalização do Min-Max Scaler.

O método standard_scaler aplica a padronização Z-score, delegando os cálculos pesados para a classe de estatística da etapa anterior do projeto.

3.4. Codificação de Variáveis Categóricas (Módulo Encoder) O método Label Encoding isola os valores únicos para construir um dicionário de mapeamento em memória:

```
valores_unicos = []
            for valor in dados_coluna:
                if valor not in valores_unicos:
                    valores_unicos.append(valor)

            # 2. Atribui um número para cada categoria de texto
            mapeamento = {valor: indice for indice, valor in enumerate(valores_unicos)}

            # 3. Substitui os valores antigos pelos novos números na coluna
            self.dataset[col] = [mapeamento[valor] for valor in dados_coluna]

```

Figura 4. Mapeamento e substituição do Label Encoder.

O One-Hot Encoding gera novas colunas binárias, removendo sumariamente a coluna raiz ao final do processo para evitar redundância.

4. Resultados e Discussões

4.1. Validação Automatizada nos Testes Unitários Os algoritmos foram submetidos a uma suíte de testes utilizando a biblioteca `unittest`. A aprovação de 100% dos testes garantiu a remoção impecável de linhas ruidosas, além de atestar que os cálculos do *Scaler* mantêm os nulos em suas posições originais sem causar quebras, e que o *Encoder* gera novas matrizes mantendo a contagem de linhas idêntica à entrada.

```
PS C:\Users\leone\Documents\GitHub\OAT's\Pre_Processamento_Dados\dende-softhouse-preprocessin
g_vitoria> python -m unittest test_preprocessing.py
............
----------------------------------------------------------------------
Ran 12 tests in 0.001s

OK
```

Figura 5. Saída do terminal exibindo a aprovação na suíte de testes unitários.

4.2. Aplicação no Dataset Spotify A eficácia do pipeline foi comprovada em um cenário real sobre o arquivo `spotify_data.csv`. A etapa de extração mapeou adequadamente *strings* vazias ou preenchidas como `"N/A"` para o tipo nativo `None`:

- **Higienização (Missing Values):** O arquivo bruto possuía **8.582 faixas**. O processador identificou que 3.361 dessas faixas (aprox. 39%) continham dados ausentes. A operação `dropna` reduziu a base para **5.221 faixas limpas**.
- **Transformação Matemática (Scaler):** A popularidade musical, cujo pico absoluto inicial era 99, foi reescalonada perfeitamente para o espectro fechado entre **0.0 e 1.0**.
- **Tradução Textual (Encoder):** Atributos textuais da coluna *album_type* (ex: `'album'`, `'single'`) foram mapeados com sucesso para identificadores absolutos numéricos (`0` e `1`).

```
PS C:\Users\leone\Documents\GitHub\OAT's\Pre_Processamento_Dados\dende-softhouse-preprocessing
_vitoria> python analise_spotify.py

==============================
 Iniciando pipeline de Pré-processamento (Spotify)
==================================================

Sucesso! 15 colunas carregadas.
1. Total de músicas (linhas) originais no CVS: 8582
Músicas que contêm dados sujos (N/A): 3361
Restaram 5221 músicas limpas para a análise.
Valor máximo antigo da popularidade: 99
Nova escala criada! Mínimo: 0.0 | Máximo: 1.0
Primeiros tipos de álbum (Texto): ['album', 'single', 'album', 'single', 'single']
Textos convertidos para IDs numéricos: [0, 1, 0, 1, 1]

==================================================
 PIPELINE CONCLUÍDO COM SUCESSO!
==================================================
```

Figura 6. Resultado da execução do pipeline de pré-processamento no *dataset* real do Spotify.

5. Considerações Finais

Nós consideramos que o nosso objetivo de aprendizagem foi plenamente cumprido. O desenvolvimento estritamente nativo destas ferramentas consolidou nossos conhecimentos avançados em manipulação de memória e estruturas de dados em Python.

Durante o projeto, **enfrentamos como principais desafios** a manipulação direta de dicionários sem o apoio do Pandas, o que nos obrigou a criar lógicas complexas para evitar erros de índice (como o IndexError) e a implementar salvaguardas matemáticas rígidas para evitar divisões por zero durante o cálculo do desvio padrão e das amplitudes. **Como pontos de melhoria para trabalhos futuros**, nós sugerimos a otimização dos nossos laços de repetição (*loops*) para reduzir a complexidade temporal, permitindo que a nossa biblioteca processe matrizes de *Big Data* em menor tempo de execução.

Referências

Faceli, K., et al. (2021). *Inteligência Artificial: Uma Abordagem de Aprendizado de Máquina*. 2ª ed. LTC.

Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques*. 3ª ed. Morgan Kaufmann.

Python Software Foundation (2024). *Python Language Reference*. Disponível em: https://docs.python.org/3/